Book Reference: Analytics Engineering with SQL and dbt

Full Title: Analytics Engineering with SQL and dbt — Building Meaningful Data Models at Scale
Authors: Rui Machado & Hélder Russa
Publisher: O'Reilly Media, First Edition (December 2023)
ISBN: 978-1-098-14238-4
Pages: 324

> The book uses **dbt Cloud + BigQuery** for all hands-on examples. We use dbt Core + Snowflake, but all commands and concepts are identical — only connection config differs.

Quick Reference: Training Module → Book Mapping

Training Module	Reading Assignment	Code Assignment
M1 — What is dbt	Ch. 1 (pp. 1–22)	Example 1-3 (p. 21)
M2 — Project Setup	Ch. 4 pp. 135–173	Replicate dbt_project.yml + profiles.yml structure
M3 — Jinja Basics	Ch. 5 pp. 233–241	Examples 5-6 and 5-7
M4 — Materializations	Ch. 5 pp. 223–232	Examples 5-1, 5-2, 5-5
M5 — Sources & Medallion	Ch. 2 pp. 63–66 + Ch. 4 pp. 184–188	Example 6-10
M6 — Testing	Ch. 4 pp. 189–196 + Ch. 6 pp. 280–285	Examples 6-24 to 6-27
M7 — Documentation	Ch. 4 pp. 200–208	Example 6-28
M9 — Jinja & Macros	Ch. 5 pp. 236–245	Examples 5-8 to 5-21
M10 — SCD2 & Snapshots	Ch. 5 pp. 230–232	Example 5-5
Capstone	Ch. 6 pp. 253–296	Full omnichannel use case

Chapter-by-Chapter Reference

Chapter 1 — Analytics Engineering

Pages: 1–22

Key Insights:
- History of data management: data warehouses (Inmon/Kimball 1980s–90s) → Big Data/Hadoop → cloud (Redshift,

BigQuery, Snowflake) → modern data stack (Airflow, dbt, Looker)
- The analytics engineer role sits at the intersection of data engineering and data analytics — owns the transformation layer
- **ELT vs. ETL:** ELT (load first, transform in place) is the modern pattern; dbt owns the T in ELT
- Data mesh: dbt as an enabler for distributed, domain-owned data products
- Legacy processes (stored procedures, ETL tools) versus the dbt revolution: version control, testing, docs, CI/CD parity with software engineering
- dbt is open source CLI (dbt Core) or managed service (dbt Cloud); both run the same commands

Code Exercise:
- **Example 1-3 (p. 21):** Minimal dbt model — a `SELECT` with `{{ ref() }}` and `{{ config(materialized='table') }}` that sums revenue across orders. Demonstrates that every dbt model is just a `SELECT` statement.

Training Module: Pre-reading for **Module 1**. Covers the "why" of dbt at industry level and reinforces the team's motivation for adopting dbt.

Chapter 2 — Data Modeling for Analytics

Pages: 23–66

Key Insights:
- Data modeling has three phases: **conceptual** (entities + relationships), **logical** (tables, FKs, normalization), **physical** (SQL DDL for a specific DB engine)
- **Normalization forms** (1NF–5NF): reduce redundancy, enforce integrity; higher normal forms split data into more tables
- **Dimensional modeling (Kimball):** fact tables (measurable events) + dimension tables (context) → star schema; snowflake schema extends this by normalizing dimensions further
- **Star schema** vs. **snowflake schema**: star is simpler and faster to query; snowflake is more normalized and storage-efficient
- **Data Vault:** hub-satellite-link design, optimized for environments where source systems change frequently
- **Modular data models in dbt:** staging layer (clean/rename) → intermediate layer (business logic) → marts layer (facts + dims); mirrors the Bronze→Staging→Silver→Gold architecture
- **Medallion Architecture (pp. 63–66):** Bronze (raw, append-only) → Silver (clean, tested, conformed dims + facts) → Gold (aggregated, BI-ready); dbt owns Silver and Gold

Code Exercise:
- No executable code exercises in this chapter. The modeling concepts are illustrated with entity-relationship diagrams and schema designs.
- Suggested activity: sketch the star schema for the project's HubSpot data (deals, contacts, pipeline stages) using the conceptual → logical → physical workflow described on pp. 25–31.

Training Module:
- **Module 1** pre-reading: pp. 47–66 (Modular Data Models + Medallion Architecture) — gives the theoretical foundation for the layer architecture
- **Module 5** pre-reading: pp. 63–66 (Medallion Architecture Pattern section) — directly explains Bronze/Silver/Gold semantics

---

Chapter 3 — SQL for Analytics

Pages: 67–133

Key Insights:
- SQL has outlasted every generation of tooling (OLTP → Hadoop → cloud DW) because of readability, strong typing, and ecosystem ubiquity
- **DDL** (Data Definition Language): `CREATE`, `ALTER`, `DROP` — defines structure
- **DML** (Data Manipulation Language): `SELECT`, `INSERT`, `UPDATE`, `DELETE` — operates on data
- **Views:** store a query definition, not data; computed on every access; ideal for abstraction and security (mirrors dbt `view` materialization)
- **CTEs** (`WITH` clause): break complex queries into named intermediate steps; dbt ephemeral models compile to CTEs in downstream models
- **Window functions** (`RANK`, `ROW_NUMBER`, `LAG`, `LEAD`, `SUM OVER`): perform calculations across a partition without collapsing rows — essential for SCD2 and ranking analytics
- **SQL for distributed processing:** DuckDB (fast in-process analytics, no dependencies), Polars, FugueSQL — modern SQL engines beyond traditional warehouses

Code Exercises:

Example	Pages	Description
3-1 to 3-4	pp. 80–82	Create `OReillyBooks` database; `CREATE TABLE` for authors, books, categories with PKs and FKs; `ALTER TABLE` to add a column
3-5 to 3-7	pp. 83–85	`INSERT INTO` dummy data for all four tables; `SELECT *` to verify
3-8 to 3-11	pp. 85–87	`SELECT` with `WHERE`, comparison/logical operators, `LIKE`, `IN`, `BETWEEN`
(section)	pp. 87–95	`GROUP BY` + aggregate functions (`SUM`, `COUNT`, `AVG`, `MAX`, `MIN`); `HAVING` clause
(section)	pp. 96–105	`JOIN` types (INNER, LEFT, RIGHT, FULL, CROSS); multi-table queries
(section)	pp. 98–101	`CREATE VIEW` and querying views
(section)	pp. 101–104	CTEs with `WITH` — chaining intermediate result sets
(section)	pp. 105–109	Window functions: `RANK()`, `ROW_NUMBER()`, `LAG()`, `SUM() OVER (PARTITION BY ...)`
(section)	pp. 113–116	DuckDB: in-memory analytics queries, CSV and Parquet reads
(bonus)	pp. 129–133	Training ML models with SQL (Dask-SQL)

Training Module:
- **Module 3 (Jinja)** supplement: pp. 98–104 (Views + CTEs) — understand what dbt `view` and `ephemeral` materialize to
- **Module 6 (Testing)** supplement: the `GROUP BY` + `HAVING` pattern (pp. 87–90) is exactly how singular dbt tests are written
- **Module 10 (SCD2)** pre-reading: pp. 105–109 (Window Functions) — `ROW_NUMBER()` and `LAG()` are the SQL

primitives behind SCD2 logic
- **Module 12 (CI/CD)** supplement: pp. 113–116 (DuckDB) — relevant for unit testing SQL logic locally

---

Chapter 4 — Data Transformation with dbt

Pages: 135–221

Key Insights:
- **dbt design philosophy** (pp. 136–137): code-centric, modular, declarative, SQL-only, documentation-as-code, incremental builds, native integration with data platforms
- **dbt data flow:** source systems → raw/bronze → dbt staging → dbt intermediate → dbt marts; dbt owns only the transformation steps
- **dbt Cloud vs. dbt Core:** Cloud is managed (scheduling, IDE, CI); Core is open source CLI — we use Core
- **Project structure** (pp. 165–167): `models/`, `macros/`, `seeds/`, `snapshots/`, `tests/`, `analyses/`, `target/`, `dbt_project.yml`
- **YAML file organization** (pp. 168–174): one `_sources.yml` + one `_models.yml` per models directory; use `_` prefix so files sort to the top
- **`dbt_project.yml`** (pp. 170–173): sets project name, model paths, default materializations per directory; all models inherit unless overridden
- **`profiles.yml`** (pp. 172–173): lives outside repo (`~/.dbt/`); contains environment-specific DB credentials
- **Models as SELECT statements** (pp. 174–184): staging layer (rename/cast), intermediate layer (joins, business logic), marts layer (facts + dims)
- **`{{ source() }}`** (pp. 184–188): registers Bronze/raw tables; enables freshness checks, lineage tracking; required before any `source()` call in a model
- **Generic tests** (pp. 189–196): `unique`, `not_null`, `accepted_values`, `relationships` — declared in YAML alongside columns
- **Singular tests** (pp. 189–196): standalone `.sql` files in `tests/`; return rows on failure
- **Documentation** (pp. 200–208): model descriptions + column descriptions in YAML; `{{ doc('...') }}` for reusable doc blocks in `.md` files; `dbt docs generate` → `dbt docs serve`
- **dbt commands** (pp. 209–211): `dbt run`, `dbt test`, `dbt build`, `dbt compile`, `dbt debug`, `dbt source freshness`, `dbt docs generate`
- **Selection syntax** (pp. 209–211): `--select model_name`, `--select +model_name+`, `--select tag:staging`, `--select source:jaffle_shop`

Code Exercises:

Example	Pages	Description
4-1	p. 142	Query Jaffle Shop public BigQuery dataset (customers, orders, payments)
4-3	p. 165	Initial dbt project folder structure after `dbt init`
4-4	p. 169	Recommended YAML file layout per models directory
4-5	p. 170	`dbt_project.yml` model config: staging → view
4-6	p. 172	`packages.yml` — install from dbt Hub, Git, or local
4-7	p. 173	`profiles.yml` for BigQuery with service account

Example	Pages	Description
4-8 to 4-11	pp. 175–178	Write three staging models (`stg_stripe_order_payments` , `stg_jaffle_shop_customers` , `stg_jaffle_shop_orders`); run `dbt run --full-refresh`
(section)	pp. 184–188	Declare sources in `_jaffle_shop_sources.yml` ; reference with `{{ source() }}`
(section)	pp. 189–196	Write generic tests (`unique` , `not_null` , `relationships`) in `_jaffle_shop_models.yml` ; run `dbt test`
(section)	pp. 200–208	Add model and column descriptions; run `dbt docs generate && dbt docs serve`

Training Modules:
- **Module 2** (Project Setup): pp. 135–173 — `dbt_project.yml` , `profiles.yml` , project structure, the five execution phases, CLI commands
- **Module 3** (Jinja): pp. 174–184 — `{{ ref() }}` and `{{ source() }}` usage in staging models
- **Module 5** (Sources): pp. 184–188 — `sources.yml` declaration and freshness
- **Module 6** (Testing): pp. 189–196 — four built-in generic tests, YAML syntax, `dbt test`
- **Module 7** (Documentation): pp. 200–208 — descriptions, doc blocks, `dbt docs serve`

Chapter 5 — dbt Advanced Topics

Pages: 223–250

Key Insights:
- **`view`** materialization: query stored on disk, data computed at runtime — for staging layers; slow to query on large datasets
- **`table`** materialization: `DROP + CREATE TABLE` on every run — fast to query, expensive to build; for marts with heavy downstream use
- **`ephemeral`** materialization: compiles to a CTE inside downstream models; no object created in DB; use sparingly (harder to debug)
- **`incremental`** materialization (pp. 227–228): processes only new/changed rows via `MERGE INTO` ; requires `unique_key` ; use `is_incremental()` macro + `{{ this }}` in WHERE clause
- **Materialized views / dynamic tables** (pp. 229–230): DB-managed incremental refresh; Snowflake uses `materialized: dynamic_table`
- **Snapshots / SCD2** (pp. 230–232): `dbt snapshot` command; `{% snapshot %}` block in `snapshots/` folder; adds `dbt_valid_from` , `dbt_valid_to` , `dbt_scd_id` columns; `strategy: timestamp` (recommended) or `check`
- **Jinja** (pp. 233–235): `{% %}` for statements (if/for/set), `{{ }}` for expressions, `{# #}` for comments; `target.name` to distinguish dev/prod behavior; `{%- -%}` trims whitespace
- **Macros** (pp. 236–241): Jinja functions in `macros/` folder; called with `{{ macro_name(args) }}` ; use `run_query()` to execute SQL at compile time; support `return()` ; can override dbt core macros (e.g., `generate_schema_name`)
- **dbt packages** (pp. 242–245): installed via `packages.yml` + `dbt deps` ; `dbt_utils` (dbt Labs) provides `date_spine()` , `safe_divide()` , `generate_surrogate_key()` , and more
- **dbt semantic layer** (pp. 246–250): entities, dimensions, measures; MetricFlow framework; abstracts metric definitions from SQL

Code Exercises:

Example	Pages	Description
5-1, 5-2	pp. 227–228	Configure `stg_jaffle_shop_orders` as incremental with `merge` strategy + `unique_key`; add `is_incremental()` filter using `{{ this }}`
5-3, 5-4	pp. 229–230	Configure materialized view (Postgres/BigQuery) and dynamic table (Snowflake) in YAML
5-5	pp. 231–232	Create `snap_order_status_transition.sql` snapshot: `{% snapshot %}` block with `timestamp` strategy; run `dbt snapshot`
5-6	p. 233	Jinja `{% if target.name != 'prod' %}` — limit data to last 3 months in dev
5-7	pp. 234–235	Dynamic pivot using `{%- set payment_types = [...] -%}` + `{% for %}` loop to generate metric columns programmatically
5-8	p. 236	Simplest macro: `{% macro sum(x, y) %}`
5-13 to 5-17	pp. 237–241	`get_payment_types()` macro using `run_query()`; generalized `get_column_values(col, table)` macro; `limit_dataset_if_not_deploy_env()` macro called from `fct_orders`
5-18 to 5-21	pp. 243–245	Install `dbt_utils` via `packages.yml` + `dbt deps`; use `date_spine()` macro; use `safe_divide()` in a model

Training Modules:
- **Module 4** (Materializations): pp. 223–232 — full materialization coverage including incremental and snapshots
- **Module 3** (Jinja) deeper dive: pp. 233–235 — `{% if %}`, `{% for %}`, `{% set %}` in real dbt models
- **Module 9** (Jinja & Macros): pp. 236–245 — creating, calling, and parameterizing macros; installing packages
- **Module 10** (SCD2 & Snapshots): pp. 230–232 — `dbt snapshot`, SCD2 columns, strategy options

---

Chapter 6 — Building an End-to-End Analytics Engineering Use Case

Pages: 253–296

Key Insights:
- A complete, real-world analytics engineering project from blank page to production: operational DB → ETL → analytics modeling → dbt implementation → tests + docs → SQL analytics
- **Operational modeling** follows conceptual → logical → physical exactly as Chapter 2 described; physical model uses MySQL DDL with audit columns (`CREATED_AT`, `UPDATED_AT`) required for incremental CDC
- **Star schema design for omnichannel retail:** 4 dimensions (`dim_channels`, `dim_customers`, `dim_products`, `dim_date`) + 2 fact tables (`fct_purchase_history`, `fct_visit_history`)
- **Naming conventions for enterprise dbt:** `stg_` (staging), `dim_` (dimension), `fct_` (fact); column prefixes `sk_` (surrogate key), `nk_` (natural/business key), `mtr_` (metric), `dsc_` (description/text), `dt_` (date/timestamp)
- **Surrogate keys** (`sk_`) are artificial identifiers assigned at the data warehouse layer; stable even if source natural keys change; generated with `dbt_utils.generate_surrogate_key()`
- **Conformed dimensions** (`dim_channels`, `dim_customers`, `dim_date`) are shared across multiple fact tables — update once, consistent everywhere
- **Singular tests for business rules** (pp. 283–285): `HAVING` clause returns failing rows — `mtr_total_amount_gross < 0`, `mtr_unit_price > mtr_total_amount_gross`
- **Analytics with star schema** (pp. 291–295): CTEs + window functions (`RANK() OVER PARTITION BY`) enable concise business queries like "top 3 products per channel" or "top customers on mobile app"

Code Exercises:

Example	Pages	Description
6-1 to 6-3	pp. 257–259	MySQL DDL: create `OMNI_MANAGEMENT` database; `CREATE TABLE` for `customers`, `products`, `channels`, `purchaseHistory`, `visitHistory` with FKs and audit columns
6-4 to 6-8	pp. 261–264	Python ETL: `mysql.connector` → `pandas` → `pandas_gbq` to load all MySQL tables into BigQuery (extract, transform dates, load functions)
6-9 to 6-10	pp. 271–272	dbt project folder structure; `_omnichannel_raw_sources.yml` to declare all 5 raw BigQuery tables as sources
6-11 to 6-18	pp. 272–278	All staging models (`stg_channels`, `stg_customers`, `stg_products`, `stg_purchase_history`, `stg_visit_history`); all dimension models (`dim_channels`, `dim_customers`, `dim_products`, `dim_date` using `dbt_utils.date_spine`)
6-19 to 6-23	pp. 278–279	Fact models `fct_purchase_history` and `fct_visit_history`: JOIN staging to dimension surrogate keys; `COALESCE(sk, '-1')` to handle unmatched FKs; compute derived metrics
6-24	pp. 281–282	Full `_omnichannel_marts.yml`: generic tests — `unique` / `not_null` on all `sk_` PKs; `relationships` tests on all FK columns in both fact tables
6-25 to 6-27	pp. 283–285	Singular tests: `assert_mtr_total_amount_gross_is_positive.sql`; `assert_mtr_unit_price_is_equal_or_lower_than_mtr_total_amount_gross.sql`; `assert_mtr_length_of_stay_is_positive.sql`
6-28	pp. 285–290	Full YAML documentation: model descriptions + column descriptions for all dims and facts
6-29 to 6-32	pp. 291–295	SQL analytics queries against the star schema: total sales per channel (simple GROUP BY), monthly revenue trend (date dim join), top 3 products per channel (CTEs + `RANK() OVER PARTITION BY`), top customers on mobile app (CTE + `LIMIT`)

Training Modules:

- **Module 2** supplement: the dbt project setup walkthrough in pp. 271–272 is a second, more complex example of structuring a real project
- **Module 5** (Sources): Example 6-10 (pp. 271–272) is the reference for writing a `sources.yml` with multiple tables
- **Module 6** (Testing): pp. 280–285 — most complete test example in the book; covers both generic and singular tests on a full star schema
- **Module 7** (Documentation): Example 6-28 (pp. 285–290) — full documented YAML including model descriptions, column descriptions, and tests together
- **Module 10** (SCD2): the `dim_date` model using `dbt_utils.date_spine()` (pp. 276–278) demonstrates packages in practice
- **Capstone / Final Project**: Ch. 6 in its entirety is the blueprint for an end-to-end project — adapting the omnichannel schema to the project's HubSpot→deals→pipeline model makes an ideal capstone exercise

---

Reading Assignment Recommendations by Module

Module	Pre-Session Reading	Approx. Time
M1 — What is dbt	Ch. 1 in full (pp. 1–22)	30 min
M2 — Project Setup	Ch. 4 pp. 135–173 (stop before Models section)	45 min
M3 — Jinja Basics	Ch. 5 pp. 233–241 (Dynamic SQL with Jinja + intro to macros)	20 min
M4 — Materializations	Ch. 5 pp. 223–232 (all materialization types + snapshots)	25 min
M5 — Sources & Medallion	Ch. 2 pp. 63–66 + Ch. 4 pp. 184–188	15 min
M6 — Testing	Ch. 4 pp. 189–196	20 min
M7 — Documentation	Ch. 4 pp. 200–208	15 min
M9 — Jinja & Macros	Ch. 5 pp. 236–245 (macros + packages)	25 min
M10 — SCD2 & Snapshots	Ch. 5 pp. 230–232 (re-read with fresh eyes after M10 live demo)	10 min
Capstone	Ch. 6 in full (pp. 253–296)	60 min

Code Assignment Recommendations by Module

Each exercise below can be assigned as async homework. The book uses BigQuery — participants should adapt to Snowflake syntax where noted.

Module	Assignment	Book Reference	Snowflake Notes
M1	Write a dbt model that SELECTs from an orders table and computes `sum(revenue)` as `total_revenue` using `{{ ref() }}`	Example 1-3, p. 21	Identical syntax
M2	Write a `dbt_project.yml` that sets staging→view, silver→table, gold→table with correct model paths	Example 4-5, p. 170	Replace `dataset:` with `schema:`
M3	Refactor a hard-coded pivot (3 columns) into a Jinja `{% for %}` loop with a `{%- set -%}` variable list	Example 5-7, pp. 234–235	Identical syntax
M4	Convert a staging model to incremental with `merge` strategy; add `is_incremental()` filter; run both full-refresh and incremental	Examples 5-1 + 5-2, pp. 227–228	Replace `DATE_SUB` with `DATEADD`
M5	Write a complete `_sources.yml` for 3 HubSpot Bronze tables with freshness thresholds	Example 6-10, pp. 271–272	Set `database:` + `schema:` for Snowflake Bronze
M6	Write a full test suite for a fact table: PKs (`unique` + `not_null`), FKs (`relationships`), singular test for negative values	Examples 6-24 + 6-25, pp. 281–283	Identical syntax
M7	Add model description with grain statement + column descriptions for all columns in a Silver dimension	Example 6-28, pp. 285–290	Identical syntax

Module	Assignment	Book Reference	Snowflake Notes
M9	Write a `generate_surrogate_key()` wrapper macro that calls `dbt_utils.generate_surrogate_key()` ; use it in a staging model	Examples 5-14 + 5-15, pp. 238–239	Identical syntax
M10	Create a snapshot for a HubSpot deals status column using `timestamp` strategy	Example 5-5, pp. 231–232	Replace `target_schema` with Snowflake schema name
Capstone	Build a mini star schema from the project's HubSpot data: 2 dims + 1 fact; write all tests and docs; run `dbt build`	Full Ch. 6	Adapt BigQuery project/dataset → Snowflake database/schema